

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Санкт-Петербургский политехнический университет Петра
Великого»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчет
Структуры данных
Летняя научно-исследовательская работа

Выполнил:

студент группы 5130201/50301 _____ Ташлыкова Е.В.

Принял:

Лектор _____ Глазунов В.В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Особенности реализации	5
2.1 Установка и настройка среды виртуализации	5
2.2 Работа в командном интерпретаторе	7
2.3 Разработка статических и динамических библиотек	10
2.4 Исследование разделяемых библиотек	12
2.5 Двоичные файлы и визуализация в псевдографике	13
3 Результаты работы и эксперименты	14
Заключение	14
Список используемых источников	15

Введение

В рамках летней практики было необходимо выполнить работу с использованием ОС GNU/Linux Debian 12. с использованием таких инструментов и технологий, как VirtualBox, SSH, CMake, ncurses, zlib. Также редактор кода Visual Studio и Nano, задания из индивидуального задания были написаны на языке программирования Си.

Практика направлена на закрепление навыков работы в Linux-среде, разработки на языке Си, управления зависимостями и сборкой проектов, а также на исследование производительности программ при различных подходах к линковке.

1 Постановка задачи

Для того, чтобы считать работу выполненной необходимо закрыть следующие задачи:

1. Установить и настроить виртуальную среду Debian 12.14;
2. Обучиться основам работы в командном интерпретаторе;
3. Разработать функцию проверки слова на палиндром в итеративном и рекурсивном вариантах;
4. Сборка статической и динамической библиотек, реализация функции через `dlopen`;
5. Провести тестирование скорости при различных флагах оптимизации (`O0`, `O1`, `O2`, `Os`);
6. Провести исследование зависимостей с `ldd`;
7. Создание структуры `PRICE`;
8. Реализация заголовка базы данных;
9. Создание функции поиска средней цены товара с максимальным количеством в заданной категории;
10. Визуализировать структуру в псевдографике `ncurses`;
11. Описать проделанную работу в итоговом отчете.

2 Особенности реализации

2.1 Установка и настройка среды виртуализации

VirtualBox – это кроссплатформенное программное обеспечение для виртуализации, которое позволяет запускать несколько операционных систем (ОС) одновременно на одном физическом компьютере. В качестве ISO-образа использовался *debian – 12.14.0 – arm64 – netinst*.

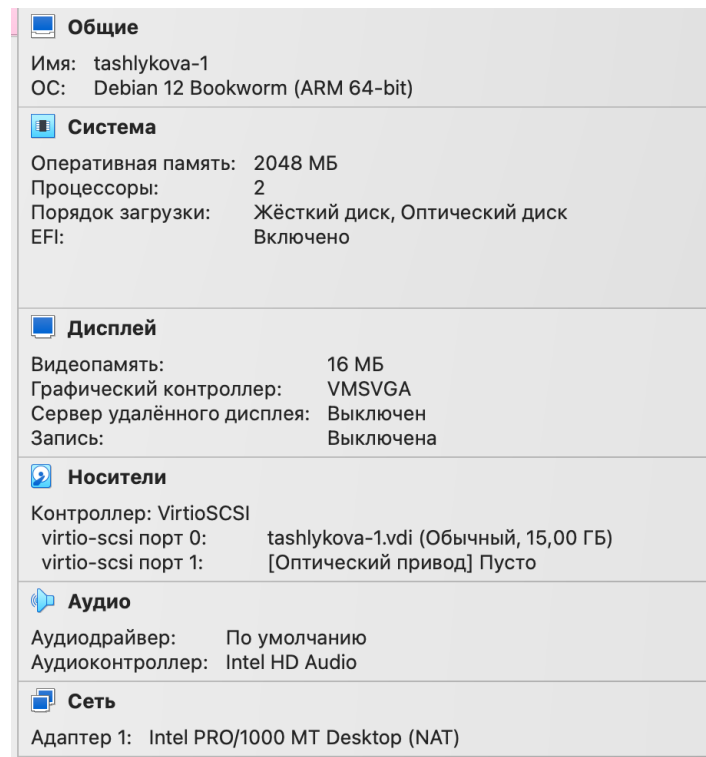


Рис. 1: Детали виртуальной машины

После загрузки системы на терминале `tty1` отображается приглашение к вводу имени пользователя и пароля.

```
Debian GNU/Linux 12 tashlykova-1 tty1
tashlykova-1 login: tashlykova
Password:
Linux tashlykova-1 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 10 17:07:40 MSK 2026 on tty1
tashlykova@tashlykova-1:~$ _
```

Рис. 2: Приглашение tty1

Также в виртуальной машине была выполнена настройка проброса портов (Port Forwarding):

- Тип подключения: NAT
- Протокол: TCP;
- Адрес хоста: 127.0.0.1;
- Порт хоста: 2222;
- Порт гостя: 22.

Это позволило подключаться к гостевой ОС через SSH.

Имя	Протокол	Адрес хоста	Порт хоста	Адрес гостя	Порт гостя
tcp_2222_22	TCP	127.0.0.1	2222		22

Рис. 3: Проброс портов

Подключение к виртуальной машине выполнено с помощью встроенного SSH-клиента. Для беспарольной SSH-аутентификации необходимо сгенерировать пару ключей, а публичный ключ скопировать на виртуальную машину.

```
ssh-keygen -t rsa -b 4096
ssh-copy-id -p 2222 tashlykova@127.0.0.1
ssh -p 2222 tashlykova@localhost
```

Благодаря этим действиям подключение к ВМ осуществляется без ввода пароля.

2.2 Работа в командном интерпретаторе

Интерпретатор предоставляет большое количество команд для работы. С помощью *man* я ознакомилась со справкой команд из списка задания, они приведены в таблице [1](#).

Таблица. 1: Основное назначение команд

Команда	Пояснение
whoami	Вывод имени текущего пользователя
date	Вывод текущей даты и времени
uptime	Время работы системы и нагрузка
df	Информация о свободном месте на дисках
cp	Копирование файлов
mv	Перемещение/переименование файлов
touch	Создание пустого файла
rm	Удаление файлов
stat	Информация о файле (размер, права, время)
file	Определение типа файла
find	Поиск файлов по критериям
cat	Вывод содержимого файла
hexdump	Просмотр файла в шестнадцатеричном виде
less	Постраничный просмотр файла
time	Измерение времени выполнения команды

```

[tashlykova@tashlykova-1:~/task1/build$ whoami
tashlykova
[tashlykova@tashlykova-1:~/task1/build$ cd ..
[tashlykova@tashlykova-1:~/task1$ cd ..
[tashlykova@tashlykova-1:~$ whoami
tashlykova
[tashlykova@tashlykova-1:~$ date
Sat Jun 20 03:41:15 PM MSK 2026
[tashlykova@tashlykova-1:~$ uptime
 15:41:34 up 19:16,  2 users,  load average: 0.00, 0.00, 0.00
[tashlykova@tashlykova-1:~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
udev              960388          0    960388   0% /dev
tmpfs             202300          504    201796   1% /run
/dev/sda2        13873468 2187960  10958980  17% /
tmpfs            1011488          0    1011488   0% /dev/shm
tmpfs              5120           0         5120   0% /run/lock
/dev/sda1         524008         6208    517800   2% /boot/efi
tmpfs            202296          0    202296   0% /run/user/1000
[tashlykova@tashlykova-1:~$ touch test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 test.c tree.sh
[tashlykova@tashlykova-1:~$ rm test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 tree.sh
[tashlykova@tashlykova-1:~$ whereis main.c
[tashlykova@tashlykova-1:~$ cat task1/src/palindrome.h
// palindrome.h
#ifndef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

```

Рис. 4: Работа в командном интерпретаторе

С помощью *history* можно просмотреть всю историю команд. Зафиксируем три последние:

```

chmod +x scripts/*.sh
./scripts/analyze_libs.sh
./scripts/test_optimization.sh

```

Далее было необходимо исследовать зависимости между программой командного интерпретатора */bin/bash* и динамическими библиотеками операцион-

ной системы при помощи команды `ldd`. Для этого построим рекурсивное дерево зависимостей. Для наглядности был разработан скрипт *tree.sh*.

```
# tree.sh
function print_deps() {
    local file=$1
    local indent=$2

    if [ ! -f "$file" ]; then
        echo "${indent}[НЕ НАЙДЕНО: $file]"
        return
    fi

    # Получаем зависимости через ldd
    ldd "$file" 2>/dev/null | grep "=> /" | awk '{print $3}' | while read lib; do
        echo "${indent}|__ $(basename $lib)"
        # Рекурсивно вызываем функцию с увеличенным отступом
        print_deps "$lib" "${indent}  "
    done
}

print_deps "$1" ""
```

Все программы зависят от `libc.so.6` – стандартной библиотеки языка Си. Больше всего зависимостей у *bin/ls*.

```

[tashlykova@tashlykova-1:~$ ./tree.sh /bin/bash
└─ libtinfo.so.6
   └─ libc.so.6
└─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /bin/ls
└─ libselinux.so.1
   └─ libpcre2-8.so.0
      └─ libc.so.6
└─ libc.so.6
└─ libc.so.6
└─ libpcre2-8.so.0
   └─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /usr/bin/grep
└─ libpcre2-8.so.0
   └─ libc.so.6
└─ libc.so.6
tashlykova@tashlykova-1:~$ █

```

Рис. 5: Деревья зависимостей

2.3 Разработка статических и динамических библиотек

В индивидуальном задании №1 было необходимо создать библиотеку с реализацией функции, которая проверяет является ли указанное слово палиндромом, при этом она должня быть реализована различными способами: с рекурсией и без рекурсии, также в реализации запрещалось использовать библиотечные функции. Для этого я написала две основные функции *palindrome*, *palindrome_recursive* – итеративная и рекурсивная реализация, а так же одну вспомогательную функцию *string_len*, для вычисления длины строки.

```

#ifdef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

```

#endif

Алгоритм *palindrome*: сравниваются символы с начала и конца строки, постепенно сдвигаясь к центру. При обнаружении несовпадающей пары возвращается false. Время работы $O(n/2)$, память $O(1)$.

Алгоритм *palindrome_recursive*: базовый случай – пустая строка или строка из одного символа являются палиндромами. Иначе сравниваются первый и последний символы строки; при совпадении выполняется рекурсивный вызов для подстроки без крайних символов. Время работы $O(n)$, память $O(n)$.

Исходный код приведен в Приложении А.

Структура проекта:

```
task1
----CMakeLists.txt
----src/
    |---CMakeLists.txt
    |---palindrome.c
    |---palindrome.h
----tests/
    |---CMakeLists.txt
    |---test_palindrome.c
    |---test_dlopen.c
----scripts/
    |---analyze_libs.sh
    |---test_optimization.sh
```

Статическая библиотека – это коллекция объектных файлов (.o), объединенных в один архивный файл. Во время компиляции программы, использую-

щей статическую библиотеку, компоновщик (linker) копирует весь необходимый код из библиотеки в исполняемый файл. Таким образом, исполняемый файл становится самодостаточным и не зависит от наличия библиотеки в системе во время выполнения.

Динамическая (разделяемая) библиотека – это коллекция скомпилированного кода, который загружается в память программы во время запуска. Исполняемый файл содержит только ссылки на функции и данные этой библиотеки, а не их копии. Это позволяет нескольким программам использовать одну и ту же копию библиотеки в памяти.

При обычной компоновке с динамической библиотекой, ОС автоматически загружает все необходимые библиотеки при запуске программы. Если какая-либо из них отсутствует, программа не запустится.

Однако существует еще более гибкий механизм работы с динамическими библиотеками – их динамическая загрузка во время выполнения (runtime loading). Runtime loading предоставляет программисту полный контроль над процессом, это означает, что программа может решить, какую библиотеку загрузить и какие функции из нее использовать, не на этапе компиляции, а уже в процессе своей работы. Для реализации этого механизма в Unix/Linux используется набор функций, предоставляемых библиотекой dl (dynamic linking library)

Сборка библиотек происходит благодаря сборочному файлу *src/CMakeLists.txt*.

2.4 Исследование разделяемых библиотек

Как происходит тестирование функций *palindrome* и *palindrome_recursive*:

1. Генерируются случайные строки заданной длины;

2. Для каждой строки вычисляется длина с помощью *string_len*;
3. Выполняется проверка на палиндром (итеративная или рекурсивная);
4. Измеряется процессорное время с помощью *clock()*;
5. Результат усредняется по количеству итераций.

Тестирование проходит для каждого типа линковки библиотеки. Также дополнительно разработан скрипт для автоматического тестирования всех флагов оптимизации (O0, O1, O2, Os).

2.5 Двоичные файлы и визуализация в псевдографике

Следующий этап научно-исследовательской работы – разработка программы с использованием псевдографической библиотеки *ncurses*.

Ncurses («new curses») – это библиотека предназначенная для управления вводом-выводом на терминал. Она позволяет работать с виртуальным экраном, создавать и управлять несколькими окнами на одном терминале, каждое со своим содержимым и атрибутами.

В рамках задания было необходимо реализовать структуру с именем *PRICE*, содержащую следующие поля:

- Название категории (строка *Си*).
- Название товара (строка *Си*).
- Стоимость товара в рублях (вещественный тип).
- Количество товаров в наличии (целочисленный тип).

А так же создать структуру описывающую заголовок базы данных:

- Сигнатура формата файла (4 байта) – содержит первые 4 буквы фамилии латиницей.
- Номер транзакции (4 байта) – целое число, которое должно увеличиваться при каждом обращении к файлу, как при чтении, так и при записи.
- Количество структур в файле (4 байта) – количество структур данных из п.1 следующих за заголовком.
- Контрольная сумма (4 байта) – контрольная сумма всех данных следующих за заголовком по алгоритму CRC-32 вызываемая из библиотеки zlib.h

Далее представлена структура *PRICE*, а так же структура для заголовка базы данных *DB_HEADER*.

```
typedef struct
{
    char *category;
    char *good;
    double price;
    unsigned int num;
} PRICE;

typedef struct
{
    char signature[4];
    unsigned int tr_num;
    unsigned int st_count;
    unsigned int crc32;
} DB_HEADER;
```

3 Результаты работы и эксперименты

Заключение

Список используемых источников

- [1] Петросян Л. А., Зенкевич Н. А., Семина Е. А. Теория игр : учеб. пособие для ун-тов. – Москва : Высш. шк. : Книжный дом «Университет», 1998. - 304 с. - ISBN 5-06-001005-8.
- [2] [Электронный ресурс] : <https://www.oracle.com/virtualization/virtualbox/> (дата обращения: 22.05.2026).
- [3] Adiaty, A. «Planning and Tracking Projects with GitHub’s Projects Tool» [Электронный ресурс] : <https://dev.to/adiatiayu/planning-and-tracking-projects-with-githubs-projects-tool-572l> (дата обращения: 23.05.2026).